

SafetYSec - Kotlin



Autores

David Martins Correia – 2023140410

Tiago Gouveia Filipe – 2019112767

Coimbra, 04/01/2026



**Instituto Superior
de Engenharia**

Politécnico de Coimbra

ÍNDICE:

Índices de figuras	1
Introdução	2
Soluções Técnicas Implementadas	3
Gestão de Estado e Interface Reativa (Compose).....	3
Persistência e Monitorização: Foreground Service.....	3
Detecção de Incidentes (Acelerómetro)	3
SDK Google Maps e Geofencing Customizado	4
Pipeline de Vídeo e Evidência (CameraX)	4
Idiomas.....	4
Interface de Utilizador	5
Conclusão	10

ÍNDICES DE FIGURAS

Fig. 1 – Página de registo.....	5
Fig. 2 – Página de login.....	5
Fig. 3 – Editar perfil	6
Fig. 4 – Página de Protegido	7
Fig. 5 – Regas de monitorização.....	7
Fig. 6 – Cancelar alerta através do código	8
Fig. 7 – Definição de zona segura do protegido	9
Fig. 8 – Página principal do Monitor	9

INTRODUÇÃO

O presente relatório descreve o desenvolvimento da aplicação móvel SafetYSec, uma solução tecnológica dedicada à monitorização e assistência de utilizadores em situações de vulnerabilidade, realizada no âmbito da unidade curricular de Arquitetura de Móveis. O objetivo principal do projeto consiste na criação de um ecossistema digital que estabelece uma ligação em tempo real entre dois perfis distintos — o Protegido e o Monitor — garantindo uma resposta rápida perante eventos de emergência como quedas, acidentes ou desorientação geográfica.

A aplicação foi desenvolvida de forma nativa para a plataforma Android utilizando a linguagem de programação Kotlin e a framework Jetpack Compose, tirando partido da sua arquitetura reativa e declarativa para gerar interfaces. Para além da gestão de perfis e autenticação, o projeto integra as seguintes funcionalidades:

Sincronização Cloud em Tempo Real: Utilização do Firebase Firestore para a propagação imediata de alertas e gestão de dados estruturados entre dispositivos.

Serviços de Monitorização Contínua: Implementação de um Foreground Service que assegura a execução de processos críticos (recolha de localização e análise de sensores) em segundo plano, garantindo a resiliência do sistema.

Deteção Automática de Incidentes: Processamento de dados provenientes do acelerómetro para a identificação de padrões de queda e impactos de alta energia (acidentes).

Integração de Mapas e Geofencing: Utilização do Google Maps SDK para a visualização de trajetórias e configuração dinâmica de "Zonas Seguras", permitindo o controlo de perímetro geográfico.

Gestão de Evidência Multimédia: Integração da biblioteca CameraX para a captura e upload automático de evidências de vídeo para o Firebase Storage em situações de alerta (não funcional).

Privacidade e Segurança: Implementação de janelas de autorização temporal para monitorização e autenticação multifator (MFA) para proteção de dados sensíveis.

SOLUÇÕES TÉCNICAS IMPLEMENTADAS

Nesta secção, detalham-se as opções de implementação que permitiram cumprir os requisitos funcionais, focando-se na robustez e na experiência do utilizador.

Gestão de Estado e Interface Reativa (Compose)

A interface foi construída utilizando o **Jetpack Compose**.

- **MutableState e Recomposição:** O estado da UI é gerido através de `MutableState` dentro de `ViewModels`. Sempre que um dado é alterado no `Firebase` (ex: um novo alerta), o estado é atualizado e o `Compose` desencadeia uma alteração inteligente, redesenhando apenas os componentes afetados.
- **LaunchedEffect:** Fundamental para a sincronização assíncrona. Utilizou-se o `LaunchedEffect` para iniciar listeners do `Firestore` de forma vinculada ao ciclo de vida dos ecrãs. Isto garante que, ao navegar para fora de um ecrã de monitorização, a coroutine associada e o listener do `Firebase` sejam automaticamente cancelados.

Persistência e Monitorização: Foreground Service

Para cumprir o requisito de monitorização contínua, implementou-se um **Foreground Service**.

- **Notificação Persistente:** O serviço mantém uma notificação visível, o que eleva a prioridade do processo perante o sistema operativo, evitando o encerramento por gestão de memória.
- **Sensor Manager & GPS:** O serviço encapsula a lógica do `SensorManager` para monitorizar o acelerómetro para obter coordenadas geográficas. Esta centralização permite que a lógica de deteção de quedas e saídas de zona segura corra de forma independente da interface gráfica.

Deteção de Incidentes (Acelerómetro)

A deteção de quedas e acidentes baseia-se no processamento de vetores de aceleração linear.

- **Algoritmo de Impacto:** A principal dificuldade técnica residia no facto de ambos os eventos gerarem picos de força G elevados. A solução consistiu na implementação de uma análise baseada em janelas temporais. A diferenciação entre queda e acidente é realizada através da análise da força G. Se existir um período de baixa força e nos próximos segundos existir um elevado impacto o alerta é considerado como queda, por outro lado se não existir a parte queda livre o alerta vai ser considerado com acidente devido ao elevado impacto.
- **Filtros de Segurança:** Para evitar falsos positivos, após a deteção de um impacto, o sistema inicia uma contagem decrescente. O alerta só é enviado para o Monitor se o utilizador não cancelar a operação através do seu PIN de segurança.

SDK Google Maps e Geofencing Customizado

A visualização espacial utiliza o **Google Maps SDK for Android**.

- **Visualização:** Marcadores dinâmicos indicam a última posição conhecida do Protegido.
- **Geofencing:** Em vez de utilizar as APIs de geofencing de sistema implementou-se uma validação manual no serviço. O Monitor define uma posição e um raio no mapa. O serviço calcula a distância em cada atualização de GPS; se a distância exceder o raio e estiver dentro do horário permitido, o alerta é disparado.

Pipeline de Vídeo e Evidência (CameraX)

O sistema de pânico exige a recolha de provas visuais conforme estipulado nos requisitos.

- **Captura Automática:** Utilizou-se a biblioteca **CameraX** para automatizar a gravação de 30 segundos de vídeo. A gravação ocorre em baixa resolução (SD) para garantir que o ficheiro seja leve o suficiente para ser enviado.

Idiomas

A aplicação foi estruturada para suportar múltiplos idiomas (Português e Inglês) através da separação de recursos.

- **Strings Resources:** Todas as etiquetas, mensagens de erro e descrições de acessibilidade residem em ficheiros strings.xml.

INTERFACE DE UTILIZADOR

O registo na aplicação é efetuado com uso da autenticação firebase, efetuando um registo e indicando um código de segurança, futuramente poderia ser trocado por Google Authenticator.

The screenshot shows the 'Criar Nova Conta' screen of the SafetYSec application. At the top, there is a dark blue header with the text 'SafetYSec' and a small icon. Below the header, the title 'Criar Nova Conta' is centered. There are three input fields: 'Nome Completo', 'Email', and 'Palavra-passe'. Below these fields, there are two radio buttons: 'Monitor' (selected) and 'Protegido'. At the bottom, there is a large blue button labeled 'Registar' and a link that says 'Já tem conta? Faça Login'.

Fig. 1 – Página de registo

The screenshot shows the 'Entrar' screen of the SafetYSec application. At the top, there is a dark blue header with the text 'SafetYSec' and a small icon. Below the header, the title 'Entrar' is centered. There are two input fields: 'Email' and 'Palavra-passe'. Below these fields, there is a large blue button labeled 'Entrar'. At the bottom, there is a link that says 'Não tem conta? Registar'.

Fig. 2 – Página de login

O utilizador tem a possibilidade de fazer a alteração dos seus dados pessoais sendo feito um update aos dados do utilizador na base de dados.

SafetYSec

O Meu Perfil

Nome

Monitor

Email (Apenas leitura)

monitor@isec.pt

Segurança

Código de Segurança (4 dígitos)

Nova Password (Vazio para manter)

Guardar Alterações

Fig. 3 – Editar perfil

O utilizador que esta a ser monitorizado, ou seja, protegido, pode habilitar ou desabilitar o envio de alertas. Para realizar a associação de um monitor tem de ser fornecido o código de associação que é gerado de forma aleatória, após o monitor aceitar o pedido aparecerá uma lista com os seus monitores. Os protegidos têm um menu de configurações onde indica o horário e algumas definições a ter em conta para disputar alertas.



Fig. 4 – Página de Protegido

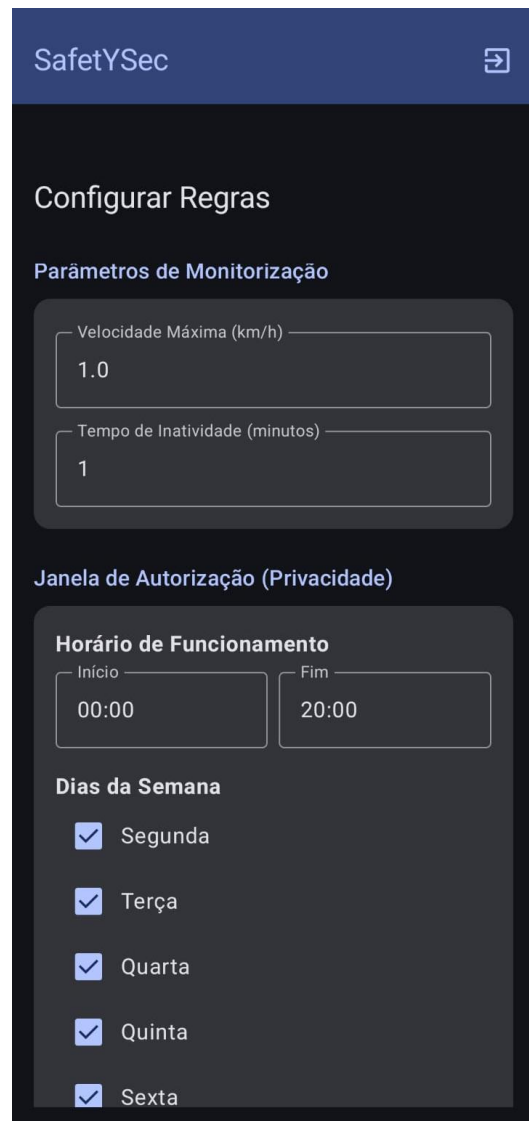


Fig. 5 – Regas de monitorização

O protegido pode acionar um alerta de pânico através do botão disponibilizado, ou alertas podem ser enviados através do funcionamento da aplicação como a deteção de uma queda ou um acidente. Para evitar falsos alertas o utilizador tem a opção de cancelar o alerta com o código definido no registo.

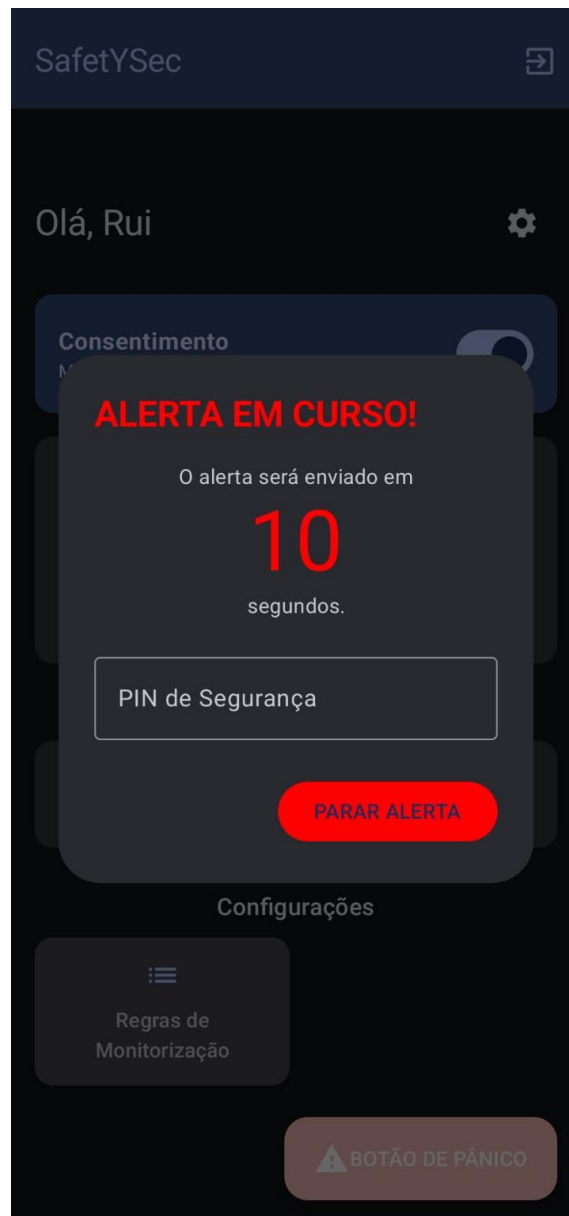


Fig. 6 – Cancelar alerta através do código

O painel do monitor é possível associar um protegido através do código fornecido e apresenta os seus protegidos e os alertas dos mesmos. É possível definir uma zona de segurança para cada protegido que pode ser traduzido para um alerta.

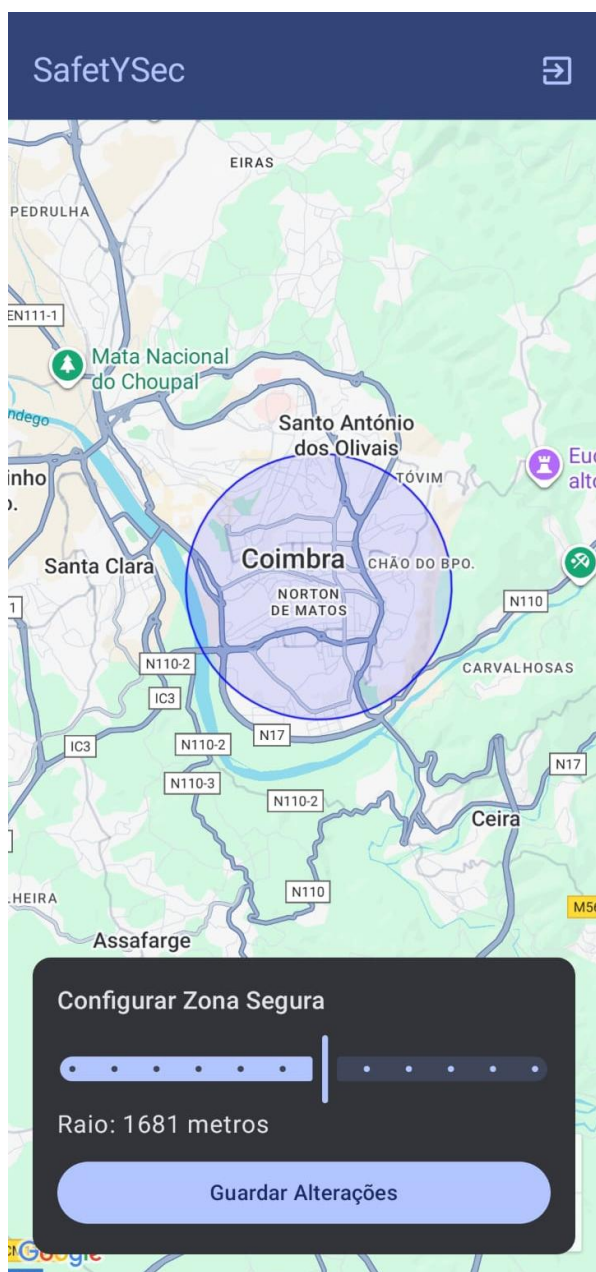


Fig. 7 – Definição de zona segura do protegido



Fig. 8 – Página principal do Monitor

CONCLUSAO

O desenvolvimento do SafetYSec permitiu a consolidação de conhecimentos avançados no domínio do desenvolvimento de aplicações móveis nativas para Android, com particular incidência na gestão de serviços críticos e na sincronização de dados em tempo real.

A utilização da arquitetura reativa com Jetpack Compose, aliada à infraestrutura Firebase, revelou-se uma decisão acertada. Esta combinação permitiu criar uma interface fluida que reage instantaneamente a eventos críticos, como o disparo de um alerta de pânico ou a deteção automática de uma queda.



**Instituto Superior
de Engenharia**

Politécnico de Coimbra